

Web Worker / Worker Threads

Anwendungsbereich

Web Worker werden im Browser verwendet, wenn rechenintensive Aufgaben die Benutzeroberfläche blockieren würden.

Da JavaScript im Browser normalerweise nur einen Thread nutzt, kann eine große Berechnung dazu führen, dass Buttons nicht mehr reagieren oder die Seite „einfriert“.

Ein Web Worker läuft in einem separaten Thread und übernimmt solche Aufgaben, sodass die UI weiterhin flüssig bleibt.

Wann machen WebWorker wenig sinn?

Nicht sinnvoll sind Worker für einfache Aufgaben oder normale API-Requests, da der Aufwand dafür meist zu groß ist.

Ein Web Worker läuft **in einer eigenen Umgebung**, getrennt vom normalen JavaScript im Browser:

- kein Zugriff auf `window`
- kein Zugriff auf DOM (`document`)
- eigenes globales Objekt → `self`

Ziel der Übung

In dieser Übung klickt der Nutzer auf einen Button, um eine große Berechnung zu starten. Anstatt die Berechnung direkt im Hauptthread (`main.js`) auszuführen, wird sie an einen Web Worker ausgelagert. Der Worker führt die Schleife unabhängig vom UI-Thread aus und sendet das Ergebnis zurück. Während die Berechnung läuft, bleibt die Seite weiterhin bedienbar. Dadurch wird deutlich, wie Web Worker helfen, Performance-Probleme im Frontend zu vermeiden.



Projekt Struktur

/webworker-demo

index.html

main.js

worker.js

index.html

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8" />
  <title>Web Worker Demo</title>
</head>
<body>

  <h1>Web Worker Demo</h1>

  <button id="start">Start Berechnung</button>
  <p id="output">Ergebnis: -</p>

  <script type="module" src="./main.js"></script>
</body>
</html>
```

Main.js

```
const button = document.getElementById('start');
const output = document.getElementById('output');

const worker = new Worker('./worker.js');

button.addEventListener('click', () => {
  output.textContent = 'Berechne...';

  worker.postMessage(1_000_000_000);
});

worker.onmessage = (e) => {
  output.textContent = 'Ergebnis: ' + e.data;
};
```

Worker.js

In dem unteren Beispiel ist `self` enthalten.

`self` ist das **globale Objekt innerhalb des Web Workers**.

`Self.onmessage` => empfängt Nachrichten vom Hauptthread

`self.postMessage` => sendet Daten zurück an den Hauptthread

```
self.onmessage = function (e) {  
  let sum = 0;  
  
  for (let i = 0; i <= e.data; i++) {  
    sum += i;  
  }  
  
  self.postMessage(sum);  
};
```

alternative Schreibweise

```
onmessage = (e) => {  
  postMessage(e.data * 2);  
};
```